برنامهٔ مرحله‌ای دومغزی — ممیزی و وضعیت

این سند فارسی، پاسخ کوتاه و قابل‌آزمون به برنامهٔ پنج‌فازی مالک است: هر بند برنامه به بخشی از
همین مخزن نگاشت شده، برای هر بند دستوری آمده که خودتان می‌توانید اجرا کنید، و هر چیزی که
هنوز ساخته نشده با نام و شرطِ شروع ثبت شده است. سند انگلیسیِ همرده ROADMAP.md است؛ اگر
شکافتی دیدید، ROADMAP.md و این سند باید یک چیز بگویند.

نسخهٔ قابل دانلود: second-brain-plan-fa.pdf

عددی که اندازه‌گیری نشده باشد، عددی نیست که کسی اثباتش کرده باشد. برای همین در این سند
عددِ تایپی کم می‌بینید و به‌جایش دستور می‌بینید: عددها را گیتِ «اعداد اسناد» در
.از خود پوشه می‌خواند python scripts/health.py

۱) خلاصهٔ وضعیت

فاز برنامه
- وضعیت
- کجای مخزن
- اثبات

۱. ممیزی و تثبیت
- انجام شد، ادامه‌دار
- 00_START_HERE/CHARTER_AUDIT.md، 02_WIKI/reference_maps/superagi-to-hades-map.md
- python scripts/health.py؛ python -m unittest discover -s tests

۲. بازآرایی و هستهٔ دانش
- انجام شد
- 15_SYSTEM/scripts/backend/sources.py، scripts/kb.py، backend/learning.py
- tests/test_sources.py، tests/test_phase27_repo_corpus.py

۳. هارنس‌ها و همگام‌سازی
- انجام شد، همگام‌سازی دستی
- 15_SYSTEM/scripts/mcp_server.py، backend/source_sync.py، backend/access.py
- tests/test_phase22_harness_bridge.py، tests/test_source_sync.py، tests/test_access_boundary.py

۴. یادگیری و خودبهبوددهی
- انجام شد؛ فاین‌تیون برنامه‌ریزی‌شده
- backend/learning.py، backend/retrieval_eval.py، scripts/base.py
- tests/test_learning.py، tests/test_retrieval_golden.py، tests/test_phase20_known_good_base.py

۵. اکوسیستم و اتوماسیون
- انجام شد
- backend/capabilities.py، backend/install_skill.py، scripts/loop.py، backend/schedules.py
- tests/test_capabilities.py، tests/test_install_skill.py، tests/test_phase28_schedule_seam.py

جمع‌بندی: چهار فاز از پنج فاز آیین‌نامه، در قالب فازهای ۳۱ تا ۳۵ نقشهٔ راه، واقعاً ساخته و

تست‌گیت شده‌اند. مورد باقی‌مانده، خط لولهٔ آماده‌سازی داده و آزمایش فاین‌تیون است که خودِ برنامه
آن را به «فاز بعد» موکول کرده — در بخش ۸ همین سند طراحی و شرط شروعش آمده است.

---

۲) روش ممیزی (تا ادعا نماند)

هر هشت گیت، یک‌جا	`python scripts/health.py`
کل مجموعهٔ تست	`python -m unittest discover -s tests`
سلامت رجیستری و ناحیه‌ها	`python 15_SYSTEM/scripts/hades.py doctor`
باتری HADES، شامل دروازهٔ MCP	`python 15_SYSTEM/scripts/test_hades.py`
همان چیزی که CI از یک کلون تازه می‌خواهد	`python scripts/ci_hades_plane.py --battery`

هر ردیف جدول‌های این سند با یکی از همین دستورها یا با یک فایل تست قابل بازبینی است. اگر
عددی در متن دیدید که با خروجی این دستورها نمی‌خواند، متن اشتباه است، نه دستور.

---

۳) فاز ۱ — ممیزی و تثبیت

خواسته: رفع مانع اجرا، بررسی کد و تست‌های موجود، فهرست بخش‌های کاربردی/تکراری/ناقص، مسیر مهاجرت برای هر حذف یا جابه‌جایی، و بررسی ایده‌ها و کد SuperAGI با توجه به تناسب و مجوز.

- مانع اجرا: کل سیستم با کتابخانهٔ استاندارد پایتون ۳.۹ به بالا کار می‌کند و هیچ وابستگی
  نصب‌شدنی ندارد؛ py -3.10 توصیه‌شده است چون کنسول ویندوز با ۳.۹ متن فارسی را می‌شکند
  .(همین را می‌سنجد tests/test_toolchain.py)
- ممیزی قانون‌ها: START_HERE/CHARTER_AUDIT.md_00 آیین‌نامه را بندبه‌بند با دستور و شاهد می‌خواند و نتیجه را «محقق / جزئی / شکاف» می‌دهد.
- مهاجرت و حذف: CHANGELOG.md برای هر جابه‌جایی مسیر مهاجرت را ثبت می‌کند؛ نمونهٔ شاخص، حذف پوشه‌های قدیمی قالب در نصب و حفظ یادداشت‌های موجود کاربر است (فاز ۳۵).
- و هر بخش کد را با (MIT) مجوز SuperAGI: 02_WIKI/reference_maps/superagi-to-hades-map.md تصمیم «بگیر / رد کن» فهرست می‌کند. هیچ کدی کورکورانه کپی نشده؛ فقط الگوها و منطق
  قابل‌آزمون (صف، زمان‌بندی، پایهٔ سالم) بازنویسی شده‌اند.

شکاف باقی‌مانده: ممیزی آیین‌نامه پیش از این برنامهٔ پنج‌فازی نوشته شده بود؛ همین سند آن
شکاف را می‌بندد.

---

۴) فاز ۲ — بازآرایی و هستهٔ دانش

خواسته: تفکیک کد اجرایی، دادهٔ شخصی، مستندات، تنظیمات، تست‌ها و افزونه‌ها؛ مسیر کامل
دریافت ← ذخیره ← نمایه‌سازی ← بازیابی با ثبت منبع، زمان و نوع محتوا؛ جدا بودن دادهٔ خام از دانش استخراج‌شده و قابل بازبینی بودن هر دو.

لایه
- جای آن
- نکته

کد اجرایی
- scripts/، 15_SYSTEM/scripts/، 15_SYSTEM/scripts/backend/
- کتابخانهٔ استاندارد، بدون سرویس

دادهٔ شخصی
- brain-data/.
- بیرون از Git، پشت مرز کلون

مستندات
- ریشه ،00_START_HERE/، 02_WIKI/
- تنها مرجع قانون AGENTS.md

تنظیمات
- 15_SYSTEM/config/system.json
- نواحی، انواع، صفحه‌های حافظه

تست‌ها
- tests/
- هر قابلیت پیش از فاز بعد تست دارد

افزونه‌ها
- skills/، 15_SYSTEM/integrations/
- نصب فقط با تأیید (فاز ۵)

- دریافت و ذخیره: sources.py import یک فایل UTF-8 انتخاب‌شده را با ذخیره می‌کند. شناسه، هش محتوا به‌همراه نوع و منبع است؛ پس metadata.json و raw.txt ورود دوبارهٔ همان بایت‌ها و همان منبع، همان رکورد را برمی‌گرداند و محتوای تغییرکرده رکورد تازه می‌سازد. format، filename، origin، kind و imported_at ثبت می‌شوند.
- نمایه‌سازی: sources.py rebuild یک نمایهٔ SQLite با جدول‌های بسامد واژه می‌سازد؛ بازسازی اتمیک است و اگر یک رکورد معیوب باشد، نمایهٔ قبلی سر جایش می‌ماند.
- بازیابی: sources.py search و ابزار MCP به نام source_search، هر نتیجه را با منبع و زمان برمی‌گردانند؛ رتبه‌بندی BM25 محلی است و نسخه‌های عربی/فارسی ی و ک را یکسان می‌بیند.
- دانش استخراج‌شده: learning.py پیشنهاد درس را فقط با ارجاع به یک منبع واردشده و نقل‌قول دقیق موجود در آن می‌پذیرد؛ پیشنهاد و تأیید در brain-data/learning/. می‌مانند و فقط درس‌های تأییدشده را با شاهد برمی‌گرداند. دادهٔ خام هرگز بازنویسی نمی‌شود. recall
- دانش خودِ مخزن: scripts/kb.py نمایهٔ FTS5 روی صفحهٔ انسانی همین مخزن می‌سازد و صفحهٔ ماشین، قالب و درهای ایجنت را با همان مرز common.clone_readable() بیرون می‌گذارد.

شکاف باقی‌مانده: استخراج خودکار از گفت‌وگو تعریف نشده است (ایشیو #20). آداپتورها فقط
فایل JSONL صریحاً انتخاب‌شده را می‌خوانند، و این یک تصمیم است نه کم‌کاری: هیچ پوشه‌ای

اسکن نمی‌شود.

۵) فاز ۳ — اتصال هارنس‌ها و همگام‌سازی

خواسته: رابط مشترک برای منابع؛ اتصال‌های اولویت‌دار برای Cursor، Claude Code، Codex و بقیه؛ اتصال یا ورود انتخابی به‌عنوان روش اصلی؛ جمع‌آوری محلی فقط اختیاری؛ همگام‌سازی چنددستگاهی با تشخیص تعارض، پشتیبان و کنترل دادهٔ حساس، آزموده‌شده.

- یک رابط مشترک: SYSTEM/scripts/mcp_server.py_15 — پروتکل MCP روی stdio، بدون پورت، کلید و دیمن. scripts/sync_hosts.py --list درهای هارنس را فهرست می‌کند و AGENTS.md مرجع است؛ درهای نوشته‌شده از آن تولید می‌شوند و یک مسیر کهنه در مرجع، تنها به هر پنج ایجنت منتشر می‌شود — برای همین sync_hosts.py --check گیت است.
- آداپتورهای اولویت‌دار: claude-stream-json، codex-jsonl و cursor-stream-json. هر سه فقط متن کاربر و دستیار را نمایه می‌کنند، نشستِ قاطی‌شده را رد می‌کنند و JSONL خام انتخاب‌شده را نگه می‌دارند.
- ورود انتخابی، نه جمع‌آوری خودکار: هیچ مسیری پوشه را نمی‌گردد. sources.py batch فهرست صریح را با plan_id وابسته به محتوا پیش‌نمایش می‌دهد و بی --apply چیزی نمی‌نویسد.
ابزار MCP به نام source_import هم فقط یک مسیر نام‌برده را می‌پذیرد: اگر مسیر بیرون از مغز یا در ناحیهٔ ممنوع باشد، رد می‌شود و هیچ رکوردی نمی‌سازد.
- چنددستگاهی: sources.py transfer اول پیش‌نمایش می‌دهد، فقط رکوردهای کامل و معتبر را کپی می‌کند و سه وضعیت را گزارش می‌دهد: تازه، از قبل موجود، تعارض. تعارض یعنی دو محتوای متفاوت با یک نوع و یک منبع؛ هر دو نسخه حفظ می‌شوند و کپی فقط با --allow-conflicts انجام می‌شود. verify-copy بایت‌به‌بایت بررسی می‌کند.
- دادهٔ حساس: .brain-data/ در .gitignore است و پشت مرز کلون؛ backend/access.py خواندن .env، کلیدها، .secrets، git و نواحی runtime/lab/candidate/backup و فراداده‌ٔ کلاینت را رد می‌کند. این مرز در tests/test_access_boundary.py با نام‌های جایگزین و symlink آزموده می‌شود.

شکاف باقی‌مانده: همگام‌سازی، انتقال پوشه‌ای صریح است، نه سرویس خودکار؛ تعارض شناسایی و حفظ می‌شود ولی ادغام خودکار وجود ندارد. پشتیبان‌گیری زمان‌دار و رمزنگاری .brain-data هم ساخته نشده و در مستندات همین‌طور گفته شده است.

۶) فاز ۴ — یادگیری و خودبهبوددهی

خواسته: استخراج درس و پیشنهاد از دادهٔ مجاز، نگه‌داشتن شاهد، ارزیابی کیفیت بازیابی و عملکرد پیش و پس از تغییر، اجرای تغییر در محیط آزمایشی و ارائهٔ نتیجه با امکان

.■■■■■■

- درس با شاهد: sources.py propose پیشنهادی می‌سازد که فقط با شناسهٔ منبع و نقل‌قول دقیق
  پذیرفته می‌شود؛ evaluate دوباره چک می‌کند که منبع و نقل‌قول سر جایشان هستند و آن پرسش،
  همان منبع را بازیابی می‌کند؛ approve بدون --approve رد می‌کند و رسید می‌نویسد؛
  .فقط تأییدشده‌ها را با منبع برمی‌گرداند recall
- سنجش بازیابی: sources.py benchmark روی مجموعهٔ ارزیابی مستقل
  را می‌دهد و MRR و recall میانگین (tests/fixtures/retrieval-golden.json)
  هر پسرفت را رد می‌کند؛ دو گزارش باید مجموعهٔ مورد، حد و هش پیکرهٔ compare-benchmarks یکسان داشته باشند.
- بازگشت: scripts/base.py --save آخرین وضعیتی را ثبت می‌کند که همهٔ گیت‌ها را پاس کرده،
  را در دفتر fell_back: true به آن برمی‌گردد، اثر انگشت را اثبات می‌کند و recover--
  .می‌نویسد. بازگشت یک مکانیزم است، نه یک قصد 14_CANDIDATES/loop/recovery.jsonl
- تغییر مدل، فقط با شاهد: backend/vector.py یک درز تزریق دارد که پیش‌فرض خاموش است؛
  گیت هشتم است و روشن‌کردن آن به شاهد اندازه‌گیری‌شده و ثبت scripts/vector_check.py تأییدکننده نیاز دارد. هیچ بهبودی بدون عدد ادعا نمی‌شود.

آنچه ادعا نمی‌شود: ارزیابی ثابت نمی‌کند که یک درس «درست» یا «مفید» است؛ فقط می‌گوید
شاهدش هنوز موجود است و بازیابی آن را برمی‌گرداند. استخراج درس، تفسیر انسان یا ایجنت است.

---

۷) فاز ۵ — اکوسیستم و اتوماسیون

خواسته: فهرست مهارت‌ها، افزونه‌ها و ابزارها با مشخصات سازگاری، منشأ و مجوز؛ پیشنهاد مجموعهٔ مناسب هر سشن؛ نصب فقط پس از تأیید؛ اتوماسیون ورود داده، بازسازی نمایه، ارزیابی،
پشتیبان و گزارش خطا.

- کاتالوگ: SYSTEM/scripts/capabilities.py_15 موجودی مهارت‌ها، افزونه‌ها، رکوردهای MCP، افزونه‌های محلی و ابزارهای داخلی را با منشأ، مجوزها، فهرست تأیید و وضعیت نصب می‌دهد.
  / approved وضعیت harness-id-- پیشنهاد بر اساس وظیفه است و "..." task--
  .را می‌گوید. رکوردهای بازنشسته و منسوخ بیرون می‌مانند not_approved / unknown
- نصب با تأیید: install_skill.py سه‌گامی است: پیشنهاد (plan) ← اجرا (apply) ←
  هرگز جایگزینی. طرح، شناسهٔ مهارت، هارنس، مقصد، هش فایل‌ها و plan_id را مقید می‌کند؛ اجرا بدون --approve و با طرح کهنه یا تغییریافته رد می‌شود و در نوشتن نیمه‌کاره، نشان .می‌گذارد install-incomplete.
- اتوماسیون: scripts/loop.py یک چرخهٔ کامل با مقایسهٔ اجرای قبلی دارد و
  تصمیم «موقعش هست یا نه» را با تابع خالص می‌دهد؛ ساعت، رکورد backend/schedules.py رجیستری SCH-... است و کرون اکشن‌ها فقط زنگ هشدار. جریان‌های ورود، بازسازی نمایه،

ارزیابی و گزارش خطا در .github/workflows/ و اسکیل‌ها تعریف شده‌اند.

شکاف باقی‌مانده: نصب‌کنندهٔ کل مغز (scripts/install.py) کپی یک‌طرفه است و مسیر به‌روزرسانی ندارد؛ ارتقا یعنی پوشهٔ تازه و ادغام دستی.

---

۸) معیارهای پذیرش — بند به بند

رابط پایدار از CLI و MCP:

توانایی
- CLI
- MCP

ثبت منبع
- batch و sources.py import
- source_import

جست‌وجوی منبع
- sources.py search
- source_search

بازیابی حافظه
- scripts/memory.py --recall
- lesson_recall و memory_recall

مشاهدهٔ منشأ
- خروجی search و metadata.json هر رکورد
- همان نتیجهٔ source_search با origin و imported_at

پیشنهاد بهبود
- CANDIDATES_در 14 ،sources.py propose ،candidate_propose
- candidate_propose و lesson_propose

ارزیابی
- sources.py evaluate، benchmark، compare-benchmarks
- خواندن از مسیر CLI؛ ایجنت حق تصمیم ندارد

مدیریت قابلیت‌ها
- capabilities.py، install_skill.py
- capabilities

تست یکپارچهٔ مسیر کامل: tests/test_learning.py مسیر «ورود منبع ← بازیابی با منشأ ← استخراج پیشنهاد ← ارزیابی ← تأیید ← یادآوری» را در یک تست می‌رود؛ tests/test_batch.py پیش‌نمایش و طرح کهنه را؛ tests/test_sources.py دادهٔ تکراری و رکورد خراب را؛ tests/test_access_boundary.py دسترسی tests/test_source_sync.py تعارض همگام‌سازی را؛ tests/test_phase20_known_good_base.py ناموفق را؛ بازگشت را. اتصال هارنس، از جمله پاسخ خطا برای endpoint غایب، در tests/test_phase22_harness_bridge.py آزموده می‌شود.

تحویل هر فاز: هر فاز با اجرای واقعی، مستند نصب و استفاده، نتیجهٔ تست و گزارش محدودیت باقی‌مانده بسته شده است — ردیف‌های ROADMAP.md فاز ۳۱ تا ۳۵ و ورودی‌های CHANGELOG.md.

---

۹) فرض‌های پذیرفته‌شده

- سیستم شخصی و محلی‌محور است؛ چنددستگاهی بودن با انتقال صریح پوشه تأمین می‌شود و انتخاب
سرویس همگام‌سازی تا وقتی اندازه‌گیری توجیهش نکند، باز است.
- پردازش فقط روی داده‌ای است که کاربر انتخاب یا تنظیم کرده؛ جمع‌آوری محلی خودکار وجود ندارد
و اگر روزی اضافه شود، اختیاری و پیش‌فرض خاموش خواهد بود.
- حافظه و ارزیابی در همین فاز اول یادگیری‌اند. فاین‌تیون و خودبهبوددهی کد بعد از فراهم‌شدن داده، معیار ارزیابی و سازوکار بازگشت می‌آیند.

---

۱۰) فاز بعد — آماده‌سازی داده و آزمایش فاین‌تیون (برنامه‌ریزی‌شده)

این تنها بند برنامه است که هنوز کد ندارد، و عمداً بدون شاهد ساخته نمی‌شود. طرح پیشنهادی:

- 1. داده: خروجی از منابع واردشده و تأییدشده (source_import / sources.py import)، فقط با اجازهٔ کاربر؛ هر نمونه به source_id و بازهٔ بایت ارجاع دارد تا ارزیابی، مستقل بماند.
- 2. مجموعهٔ ارزیابی مستقل: جدا از دادهٔ آموزش، با همان قالب retrieval-golden.json که امروز benchmark می‌خواند؛ هیچ نمونه‌ای از آن در آموزش نمی‌آید.
- 3. آزمایش: مقایسهٔ پیش و پس روی همان مجموعه با compare-benchmarks؛ شرط پذیرش، بهتر
شدن قابل‌تکرار recall و MRR با حاشیهٔ مثبت است، نه یک اجرای خوش‌شانس.
- 4. بازگشت: مدل و پیکربندی قبلی از طریق scripts/base.py -- save و --recover قابل
بازگشت است؛ نتیجهٔ نهایی همراه شاهد، هزینه و محدودیت‌ها برای تأیید انسان ارائه می‌شود.
- 5. ممنوعیت: تا وقتی این چهار مورد فراهم نیست، هیچ ادعایی دربارهٔ «بهبود مدل» در مستندات یا کامیت‌ها پذیرفته نیست.

---

۱۱) محدودیت‌های باز

- تشخیص تناقض بین یادداشت‌ها ساخته نشده و تعریف اندازه‌گیری‌پذیر ندارد (ایشیو #24).
- درس‌ها با نوع صفحهٔ حافظه تایپ نمی‌شوند؛ حافظهٔ خطا و تصمیم یک دفتر مشترک دارند (ایشیو #25).
- مسیر استخراج گفت‌وگو تعریف‌نشده است؛ آداپتورها فقط فایل صریح می‌خوانند (ایشیو #20).
- ایجنت مدیر آیین‌نامه به‌عنوان رکورد ثبت نشده است (ایشیو #21).
- نصب‌کننده مسیر ارتقا ندارد و .brain-data رمزنگاری و پشتیبان‌گیری خودکار ندارد.

- همگام‌سازی، انتقال دستی است؛ تعارض شناسایی و حفظ می‌شود، ولی ادغام خودکار نیست.

اگر یکی از این موارد بسته شد، این سند و ROADMAP.md باید در یک کامیت به‌روز شوند — سند
کهنه، ادعاست.